COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE-010 Data Structures and Algorithms	**SECTION:**	CPE21S3
DATE PERFORMED:	7/21/26	**DATE SUBMITTED:**	7/21/26
NAME:	Vince Andrew Urbano	**INSTRUCTOR:**	Engr. Jimlord Quejado

Seatwork. 3.1 Creating a Simply Linklist

1. PROCEDURE OUTPUT

HOA3p1 > main.cpp > ...

```cpp
#include <iostream>
#include "simply_LL.h"

int main(){

//creating an empty list
    SimplyList<char>* head = new SimplyList<char>;
    SimplyList<char>* second = new SimplyList<char>;
    SimplyList<char>* third = new SimplyList<char>;
    SimplyList<char>* fourth = new SimplyList<char>;
    SimplyList<char>* fifth = new SimplyList<char>;

    //assign the data
    head->data = 'V';
    head->next = second;

    second->data = 'I';
    second->next = third;

    third->data = 'N';
    third->next = fourth;

    fourth->data = 'C';
    fourth->next = fifth;

    fifth->data = 'E';
    fifth->next = nullptr;

    std::cout<<"Original List"<<std::endl;
    listTraversal(head);

    std::cout<<"Inserting a new node to the head.\n";
    sllinsertHead('Q', &head);
    listTraversal(head);
    std::cout<<"Inserting a new node at the third"<<std::endl;
    sllGeneralInsert('X', head->next->next);
    listTraversal(head);

    std::cout<< "Inserting new node at the end."<<std::endl;
    sllinsertEnd('U', &head);
    listTraversal(head);

    std::cout<<"Deleting a Node"<<std::endl;
    sllDeleteNode('X',&head);
    listTraversal(head);

    std::cout<< "Deleting all the Nodes."<< std::endl;
    sllDeleteList(&head);
```

```
49      listTraversal(head);
50
51      return 0;
52  }
```

HOA3p1 > C simply_LL.h > sllGeneralInsert<T>(T, SimplyList<T>*)

```
1   #ifndef SIMPLY_LL_H
2   #define SIMPLY_LL_H
3
4   template <typename T>
5   class SimplyList{
6       public:
7           T data;
8           SimplyList<T> *next = nullptr;
9   };
10
11
12  template <typename T>
13  void listTraversal(SimplyList<T>*head){
14
15      if(head == nullptr){
16          std::cout<<"List is empty"<< std::endl;
17          return;
18      }
19      while(head != nullptr){
20
21          std::cout<<head->data;
22          if(head->next != nullptr){
23              std::cout<< "->";
24          }
25          head = head->next;
26      }
27      std::cout<<std::endl;
28  }
29
30  template <typename T>
31  void sllinsertHead(T newData, SimplyList<T>** currentHead){
32      SimplyList<T>* newNode = new SimplyList<T>;
33      newNode->data = newData;
34      newNode->next = *currentHead;
35      *currentHead = newNode;
36  }
37
38  template <typename T>
39  void sllGeneralInsert(T newData, SimplyList<T>* prevNode){
40      if(prevNode == nullptr){
41          std::cout<<"Previous Node cannot be Null"<<std::endl;
42          return;
43      }
44      SimplyList<T>* newNode = new SimplyList<T>;
45      newNode->data = newData;
46      newNode->next = prevNode->next;
47      prevNode->next = newNode;
48
```

```
49  }
50
51  template <typename T>
52  void sllinsertEnd(T newData, SimplyList<T>** head){
53
54      SimplyList<T>* newNode = new SimplyList<T>;
55      SimplyList<T>* currentNode = *head;
56      newNode->data = newData;
57      newNode->next = nullptr;
58
59      while(currentNode->next != nullptr){
60          currentNode = currentNode->next;
61      }
62
63      currentNode->next = newNode;
64
65  }
66
67  template <typename T>
68  void sllDeleteNode(T findData, SimplyList<T>** head){
69      if(*head == nullptr) return;
70
71      SimplyList<T>* currNode = *head;
72      SimplyList<T>* prevNode = nullptr;
73
74      while(currNode != nullptr  && currNode->data != findData){
75          prevNode = currNode;
76          currNode = currNode->next;
77      }
78
79      if (currNode == nullptr) return;
80
81      if (prevNode == nullptr){
82          *head = currNode->next;
83      }else{
84          prevNode->next = currNode->next;
85      }
86
87      delete currNode;
88
89  }
90
91  template <typename T>
92  void sllDeleteList(SimplyList<T>** head){
93      SimplyList<T>* current = *head;
94
```

```
95      while(current != nullptr){
96          SimplyList<T>* temp = current;
97          current = current->next;
98
99          delete temp;
100     }
101
102     *head = nullptr;
103
104 }
105
106
107
108 #endif
```

Output:

```
PS C:\msys64\ucrt64\bin> C:\Users\TIPQC\Documents\CPE_Urbano\HOA3p1\main.exe
Original List
V->I->N->C->E
Inserting a new node to the head.
Q->V->I->N->C->E
Inserting a new node at the third
Q->V->I->X->N->C->E
Inserting new node at the end.
Q->V->I->X->N->C->E->U
Deleting a Node
Q->V->I->N->C->E->U
Deleting all the Nodes.
List is empty
```

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Reflection:

During the discussion, I found it challenging at first but after trying it out using the VScode I began to understand its concepts easily. Creating our own library is very helpful for making our codes simpler and much easier to understand on our main.cpp. I will try to do more hands-on activity outside the school to practice and understand it more. The drawings on the board also helped me understand how linklist works and how one code affects another.

COMPUTER ENGINEERING

QUEZON CITY

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: _____

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: _____

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: _____

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.